

Package ‘TRexSelector’

May 20, 2026

Type Package

Title T-Rex Selector: High-Dimensional Variable Selection & FDR Control

Version 2.0.0

Description FDR-controlled variable selection for regression and classification in high-dimensional settings while controlling the false discovery rate (FDR) at a user-specified target level. The R package comes as R bindings to the TRexSelector C++ library.

Maintainer Jasin Machkour <jasin.machkour@tu-darmstadt.de>

License GPL (>= 3)

Encoding UTF-8

LazyData true

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.3

LinkingTo Rcpp, RcppEigen, BH, Rcereal

Depends R (>= 2.10)

Imports Rcpp, R6

Suggests testthat (>= 3.0.0)

SystemRequirements C++20

NeedsCompilation yes

Author Jasin Machkour [aut, cre],
Fabian Scheidt [aut],
Simon Tien [aut],
Daniel P. Palomar [aut],
Michael Muma [aut]

Contents

agglomerative_cluster	2
as.matrix.mmap_matrix	3
compute_fdp	3
compute_fdp_dense	4
compute_precision	4
compute_recall	5
compute_tpp	5

compute_tpp_dense	6
convert_to_memory_mapped	6
cut_tree	7
dim.mmap_matrix	7
DistanceMetric	8
Gauss_data	8
get_ptr	9
LinkageMethod	9
LpNormScaler	10
mmap_matrix	12
print.mmap_matrix	13
RidgeCV	13
RidgeGCV	15
TACGPSolver	17
TAFSSolver	18
TENETSolver	19
TGPSolver	21
TLARSSolver	21
TLASSOSolver	22
TMPSolver	23
TNCGMPSolver	24
TOMPSolver	25
TOOLSSolver	26
TRexBiobankScreeningSelector	27
TRexDASelector	28
TRexGVSSelector	30
TRexScreeningSelector	32
TRexSelector	34
TSolverBase	36
TStagewiseSolver	39
TSTEPWISESolver	40
ZScoreScaler	41
[.mmap_matrix	43
Index	44

agglomerative_cluster *Agglomerative Clustering*

Description

Performs hierarchical agglomerative clustering on a numeric matrix.

Usage

```
agglomerative_cluster(
  data,
  method = LinkageMethod$Ward,
  metric = DistanceMetric$Euclidean,
  use_mmap = FALSE
)
```

Arguments

data	A numeric matrix.
method	A value from LinkageMethod (e.g. LinkageMethod\$Ward).
metric	A value from DistanceMetric (e.g. DistanceMetric\$Euclidean). Default is Euclidean.
use_mmap	Whether to use memory mapping files internally. Default is FALSE.

Value

A numeric matrix representing the computed hierarchical linkage.

as.matrix.mmap_matrix *Get Zero-Copy View as Numeric Vector (Block)*

Description

Get Zero-Copy View as Numeric Vector (Block)

Usage

```
## S3 method for class 'mmap_matrix'
as.matrix(x, ...)
```

Arguments

x	mmap_matrix object
...	extra arguments including row_range and col_range

Value

Numeric matrix (copied)

compute_fdp *Compute False Discovery Proportion (FDP) using index sets*

Description

Compute False Discovery Proportion (FDP) using index sets

Usage

```
compute_fdp(selected_indices, true_support)
```

Arguments

selected_indices	Integer vector of selected indices.
true_support	Integer vector of true active indices.

Value

Numeric FDP value.

compute_fdp_dense	<i>Compute False Discovery Proportion (FDP) using dense vectors</i>
-------------------	---

Description

Compute False Discovery Proportion (FDP) using dense vectors

Usage

```
compute_fdp_dense(beta_hat, beta, eps = 1e-15)
```

Arguments

beta_hat	Numeric vector of estimated coefficients.
beta	Numeric vector of true coefficients.
eps	Numeric threshold for counting non-zeros (default: 1e-15).

Value

Numeric FDP value.

compute_precision	<i>Compute Precision using index sets</i>
-------------------	---

Description

Compute Precision using index sets

Usage

```
compute_precision(selected_indices, true_support)
```

Arguments

selected_indices	Integer vector of selected indices.
true_support	Integer vector of true active indices.

Value

Numeric precision value.

compute_recall	<i>Compute Recall using index sets</i>
----------------	--

Description

Compute Recall using index sets

Usage

```
compute_recall(selected_indices, true_support)
```

Arguments

selected_indices	Integer vector of selected indices.
true_support	Integer vector of true active indices.

Value

Numeric recall value.

compute_tpp	<i>Compute True Positive Proportion (TPP) using index sets</i>
-------------	--

Description

Compute True Positive Proportion (TPP) using index sets

Usage

```
compute_tpp(selected_indices, true_support)
```

Arguments

selected_indices	Integer vector of selected indices.
true_support	Integer vector of true active indices.

Value

Numeric TPP value.

compute_tpp_dense	<i>Compute True Positive Proportion (TPP) using dense vectors</i>
-------------------	---

Description

Compute True Positive Proportion (TPP) using dense vectors

Usage

```
compute_tpp_dense(beta_hat, beta, eps = 1e-15)
```

Arguments

beta_hat	Numeric vector of estimated coefficients.
beta	Numeric vector of true coefficients.
eps	Numeric threshold for counting non-zeros (default: 1e-15).

Value

Numeric TPP value.

convert_to_memory_mapped	<i>Convert R Matrix to System Memory Mapped Matrix</i>
--------------------------	--

Description

Maps a given R numeric matrix into a binary formatted memory-mapped file on disk efficiently.

Usage

```
convert_to_memory_mapped(mat, filename)
```

Arguments

mat	A numeric matrix.
filename	String destination file path.

Value

NULL invisibly mapped.

`cut_tree`*Cut Hierarchical Linkage Matrix*

Description

Flatten the dendrogram clustering results based on the number of requested groups.

Usage

```
cut_tree(linkage, num_orig_objs, num_clusters)
```

Arguments

<code>linkage</code>	A numeric matrix representing linkage, generated by <code>agglomerative_cluster</code> .
<code>num_orig_objs</code>	Original number of observations data rows.
<code>num_clusters</code>	Target flat group capacity.

Value

An integer vector mapping observation indexes to cluster arrays.

`dim.mmap_matrix`*Get mmap_matrix dimensions*

Description

Get `mmap_matrix` dimensions

Usage

```
## S3 method for class 'mmap_matrix'  
dim(x)
```

Arguments

<code>x</code>	<code>mmap_matrix</code> object
----------------	---------------------------------

Value

an integer vector of length 2

DistanceMetric	<i>Distance Metrics</i>
----------------	-------------------------

Description

Enumeration of supported distance metrics for hierarchical clustering. Supported metrics include: - Euclidean: Standard Euclidean distance: codes as 0. - Correlation: Correlation distance (1 - correlation): codes as 1. - Manhattan: Manhattan (L1) distance: codes as 2. - Correlation_LSH_Filter: Correlation distance with LSH-based filtering: codes as 3. - Correlation_LSH_Approx: Approximate correlation distance using LSH: codes as 4.

Usage

DistanceMetric

Format

An object of class list of length 5.

Gauss_data	<i>Toy data generated from a Gaussian linear model</i>
------------	--

Description

A data set containing a predictor matrix X with $n = 50$ observations and $p = 100$ variables (predictors), and a sparse parameter vector β with associated support vector.

Usage

Gauss_data

Format

A list containing a matrix X and vectors y , β , and support:

X Predictor matrix, $n = 50$, $p = 100$.

y Response vector.

beta Parameter vector.

support Support vector.

Examples

```
# Generated as follows:
set.seed(789)
n <- 50
p <- 100
X <- matrix(stats::rnorm(n * p), nrow = n, ncol = p)
beta <- c(rep(5, times = 3), rep(0, times = 97))
support <- beta > 0
y <- X %*% beta + stats::rnorm(n)
```



```
Gauss_data <- list(
  X = X,
  y = y,
  beta = beta,
  support = support
)
```

get_ptr

*Get the External Pointer of a MemoryMappedMatrix***Description**

Get the External Pointer of a MemoryMappedMatrix

Usage

```
get_ptr(x, ...)
```

Arguments

x A MemoryMappedMatrix / mmap_matrix object.
... unused arguments

Value

The underlying external pointer (the object itself).

LinkageMethod

*Linkage Methods***Description**

Enumeration of supported hierarchical clustering linkage methods. Supported methods include:
- Ward: Ward's minimum variance method: codes as 0. - Average (UPGMA): Average linkage: codes as 1. - Complete: Complete linkage: codes as 2. - Single: Single linkage: codes as 3. - WPGMA: Weighted pair group method with arithmetic mean: codes as 4. - Median: Median linkage: codes as 5. - Centroid: Centroid linkage: codes as 6.

Usage

```
LinkageMethod
```

Format

An object of class list of length 7.

LpNormScaler

*Lp-Norm Scaler***Description**

Scales input features to unit norm based on the chosen L1 or L2 formulation.

Methods**Public methods:**

- `LpNormScaler$new()`
- `LpNormScaler$fit()`
- `LpNormScaler$transform_inplace()`
- `LpNormScaler$inverse_transform_inplace()`
- `LpNormScaler$is_fitted()`
- `LpNormScaler$get_dropped_indices()`
- `LpNormScaler$get_means()`
- `LpNormScaler$get_scales()`
- `LpNormScaler$get_with_mean()`
- `LpNormScaler$get_with_norm()`
- `LpNormScaler$get_norm_type()`
- `LpNormScaler$save()`
- `LpNormScaler$load()`
- `LpNormScaler$clone()`

Method `new()`: Initialize the LpNormScaler

Usage:

```
LpNormScaler$new(norm_type = 2, with_mean = TRUE)
```

Arguments:

`norm_type` Integer, 1 for L1 norm, 2 for L2 norm (default: 2)

`with_mean` Logical, whether to center the data before scaling (default: TRUE)

Method `fit()`: Compute the norm parameters.

Usage:

```
LpNormScaler$fit(X, threshold = 1e-12)
```

Arguments:

`X` A numeric matrix.

`threshold` Double, lower bound for norm (default: 1e-12).

Returns: Value.

Method `transform_inplace()`: Perform standardization in-place.

Usage:

```
LpNormScaler$transform_inplace(X)
```

Arguments:

`X` A numeric matrix.

Returns: The modified matrix X.

Method `inverse_transform_inplace()`: Inverse transform to original scale.

Usage:

```
LpNormScaler$inverse_transform_inplace(X)
```

Arguments:

X A numeric matrix.

Returns: The modified matrix X.

Method `is_fitted()`: Check if scaler is fitted.

Usage:

```
LpNormScaler$is_fitted()
```

Returns: Value.

Method `get_dropped_indices()`: Get dropped indices.

Usage:

```
LpNormScaler$get_dropped_indices()
```

Returns: Value.

Method `get_means()`: Get the learned means.

Usage:

```
LpNormScaler$get_means()
```

Returns: Value.

Method `get_scales()`: Get the learned scales.

Usage:

```
LpNormScaler$get_scales()
```

Returns: Value.

Method `get_with_mean()`: Check if scaler centers data.

Usage:

```
LpNormScaler$get_with_mean()
```

Returns: Value.

Method `get_with_norm()`: Check if scaler scales data to unit norm.

Usage:

```
LpNormScaler$get_with_norm()
```

Returns: Value.

Method `get_norm_type()`: Get norm type (1 for L1, 2 for L2).

Usage:

```
LpNormScaler$get_norm_type()
```

Returns: Value.

Method `save()`: Save model state to file.

Usage:

```
LpNormScaler$save(filename)
```

Arguments:

filename String path to save output.

Returns: Value.

Method load(): Load model state from file.

Usage:

LpNormScaler\$load(filename)

Arguments:

filename String path to load input from.

Returns: Value.

Method clone(): The objects of this class are cloneable with this method.

Usage:

LpNormScaler\$clone(deep = FALSE)

Arguments:

deep Whether to make a deep clone.

mmap_matrix

Memory-Mapped Matrix Object

Description

S3 object wrapping an external pointer to a C++ MemoryMappedMatrix.

Usage

```
mmap_matrix(filename, rows, cols, mode = "readwrite")
```

Arguments

filename	Path to binary mmap file
rows	Number of rows
cols	Number of columns
mode	Access mode: 'readonly' or 'readwrite' (default)

Value

Object of classes MemoryMappedMatrix and mmap_matrix (S3 XPtr wrapper)

print.mmap_matrix	<i>Print mmap_matrix</i>
-------------------	--------------------------

Description

Print mmap_matrix

Usage

```
## S3 method for class 'mmap_matrix'
print(x, ...)
```

Arguments

x	mmap_matrix object
...	arguments passed to other methods

Value

invisibly returns x

RidgeCV	<i>Ridge Regression with Cross-Validation</i>
---------	---

Description

Performs explicit K-Fold cross-validation for Ridge Regression model selection.

Methods**Public methods:**

- `RidgeCV$new()`
- `RidgeCV$fit()`
- `RidgeCV$cv_min()`
- `RidgeCV$cv_1se()`
- `RidgeCV$index_min()`
- `RidgeCV$index_1se()`
- `RidgeCV$get_lambdas()`
- `RidgeCV$get_cv_errors()`
- `RidgeCV$get_cv_std()`
- `RidgeCV$clone()`

Method `new()`: Initialize the RidgeCV object.

Usage:

```
RidgeCV$new()
```

Method `fit()`: Fit the CV model.

Usage:

```
RidgeCV$fit(  
  X,  
  y,  
  num_folds = 10,  
  n_lambda = 100,  
  lambda_ratio = 1000,  
  seed = 0  
)
```

Arguments:

X Predictor matrix.

y Response vector.

num_folds Number of folds (default 10).

n_lambda Number of lambdas in path (default 100).

lambda_ratio Ratio of max to min lambda (default 1000).

seed Random seed for CV splits (default 0).

Returns: Value.

Method cv_min(): Get lambda that minimizes CV error.

Usage:

```
RidgeCV$cv_min()
```

Returns: Value.

Method cv_1se(): Get 1SE rule lambda.

Usage:

```
RidgeCV$cv_1se()
```

Returns: Value.

Method index_min(): Get index of minimum CV error.

Usage:

```
RidgeCV$index_min()
```

Returns: Value.

Method index_1se(): Get index of 1SE rule lambda.

Usage:

```
RidgeCV$index_1se()
```

Returns: Value.

Method get_lambdas(): Get the sequence of lambdas used.

Usage:

```
RidgeCV$get_lambdas()
```

Returns: Value.

Method get_cv_errors(): Get the CV mean squared errors for each lambda.

Usage:

```
RidgeCV$get_cv_errors()
```

Returns: Value.

Method `get_cv_std()`: Get the standard error of the CV errors.

Usage:

`RidgeCV$get_cv_std()`

Returns: Value.

Method `clone()`: The objects of this class are cloneable with this method.

Usage:

`RidgeCV$clone(deep = FALSE)`

Arguments:

`deep` Whether to make a deep clone.

RidgeGCV

Ridge Regression with GCV

Description

Efficiently solves Ridge Regression while computing Generalized Cross-Validation (GCV).

Methods

Public methods:

- `RidgeGCV$new()`
- `RidgeGCV$fit()`
- `RidgeGCV$solve()`
- `RidgeGCV$gcv()`
- `RidgeGCV$predict()`
- `RidgeGCV$predict_path()`
- `RidgeGCV$solve_path()`
- `RidgeGCV$gcv_optimal()`
- `RidgeGCV$default_lambda_grid()`
- `RidgeGCV$n_samples()`
- `RidgeGCV$n_features()`
- `RidgeGCV$rank()`
- `RidgeGCV$clone()`

Method `new()`: Initialize the RidgeGCV object.

Usage:

`RidgeGCV$new()`

Method `fit()`: Fit the SVD decomposition for the given data.

Usage:

`RidgeGCV$fit(X, y)`

Arguments:

`X` Predictor matrix.

`y` Response vector.

Returns: Value.

Method `solve()`: Solve for a specific lambda.

Usage:

```
RidgeGCV$solve(lambda_val)
```

Arguments:

`lambda_val` Numeric value for lambda.

Returns: Value.

Method `gcv()`: Compute GCV score for a specific lambda.

Usage:

```
RidgeGCV$gcv(lambda_val)
```

Arguments:

`lambda_val` Numeric value for lambda.

Returns: Value.

Method `predict()`: Predict using a specific lambda.

Usage:

```
RidgeGCV$predict(X_new, lambda_val)
```

Arguments:

`X_new` Numeric matrix for predictions.

`lambda_val` Numeric value for lambda.

Returns: Value.

Method `predict_path()`: Predict using a path of lambdas.

Usage:

```
RidgeGCV$predict_path(X_new, lambdas)
```

Arguments:

`X_new` Numeric matrix for predictions.

`lambdas` Numeric vector of lambda values.

Returns: Value.

Method `solve_path()`: Solve across a path of lambdas.

Usage:

```
RidgeGCV$solve_path(lambdas)
```

Arguments:

`lambdas` Numeric vector of lambda values.

Returns: Value.

Method `gcv_optimal()`: Find the optimal lambda.

Usage:

```
RidgeGCV$gcv_optimal(num_grid = 200, tol = 1e-08)
```

Arguments:

`num_grid` Integer, number of grid points.

`tol` Numeric, tolerance for convergence.

Returns: Value.

Method `default_lambda_grid()`: Get the default lambda grid.

Usage:

```
RidgeGCV$default_lambda_grid(num_grid = 100, ratio = 1000)
```

Arguments:

`num_grid` Integer, number of grid points.

`ratio` Numeric, ratio for lambda grid.

Returns: Value.

Method `n_samples()`: Get number of samples.

Usage:

```
RidgeGCV$n_samples()
```

Returns: Value.

Method `n_features()`: Get number of features.

Usage:

```
RidgeGCV$n_features()
```

Returns: Value.

Method `rank()`: Get rank of X.

Usage:

```
RidgeGCV$rank()
```

Returns: Value.

Method `clone()`: The objects of this class are cloneable with this method.

Usage:

```
RidgeGCV$clone(deep = FALSE)
```

Arguments:

`deep` Whether to make a deep clone.

TACGPSolver

T-ACGP Solver

Description

Terminating Approximate Conjugate Gradient Pursuit (T-ACGP) solver. Inherits all common methods from [TSolverBase](#).

Super class

[TRexSelector::TSolverBase](#) -> TACGPSolver

Methods

Public methods:

- [TACGPSolver\\$new\(\)](#)
- [TACGPSolver\\$clone\(\)](#)

Method `new()`: Create a new TACGPSolver.

Usage:

```
TACGPSolver$new(X, D, y, normalize = TRUE, intercept = TRUE, verbose = FALSE)
```

Arguments:

X Numeric matrix of predictors, or `mmap_matrix`.

D Numeric matrix of dummies.

y Numeric response vector.

normalize Logical; column-normalize X and D (default TRUE).

intercept Logical; fit an intercept (default TRUE).

verbose Logical; print progress (default FALSE).

Method `clone()`: The objects of this class are cloneable with this method.

Usage:

```
TACGPSolver$clone(deep = FALSE)
```

Arguments:

deep Whether to make a deep clone.

TAFSSolver	<i>T-AFS Solver</i>
------------	---------------------

Description

Terminating Adaptive Forward Stepwise (T-AFS) solver. The shrinkage parameter ρ interpolates between Forward Stepwise ($\rho = 1$) and LAR/LASSO ($\rho \rightarrow 0$). Inherits all common methods from [TSolverBase](#).

Super class

[TRexSelector](#): [TSolverBase](#) -> TAFSSolver

Methods

Public methods:

- [TAFSSolver\\$new\(\)](#)
- [TAFSSolver\\$clone\(\)](#)

Method `new()`: Create a new TAFSSolver.

Usage:

```
TAFSSolver$new(
  X,
  D,
  y,
  rho = 1,
  normalize = TRUE,
  intercept = TRUE,
  verbose = FALSE
)
```

Arguments:

X Numeric matrix of predictors, or `mmap_matrix`.
 D Numeric matrix of dummies.
 y Numeric response vector.
 rho Numeric shrinkage parameter in (0, 1]; default 1.0.
 normalize Logical; column-normalize X and D (default TRUE).
 intercept Logical; fit an intercept (default TRUE).
 verbose Logical; print progress (default FALSE).

Method `clone()`: The objects of this class are cloneable with this method.

Usage:

```
TAFSSolver$clone(deep = FALSE)
```

Arguments:

deep Whether to make a deep clone.

TENETSolver

T-ENET Solver

Description

Terminating Elastic Net solver (L1 + L2 regularization). Inherits all common methods from [TSolverBase](#).

Super class

[TRexSelector::TSolverBase](#) -> TENETSolver

Methods

Public methods:

- [TENETSolver\\$new\(\)](#)
- [TENETSolver\\$get_num_removals\(\)](#)
- [TENETSolver\\$get_cycling_ratio\(\)](#)
- [TENETSolver\\$get_lambda\(\)](#)
- [TENETSolver\\$clone\(\)](#)

Method `new()`: Create a new TENETSolver.

Usage:

```
TENETSolver$new(
  X,
  D,
  y,
  lambda2,
  normalize = TRUE,
  intercept = TRUE,
  verbose = FALSE
)
```

Arguments:

X Numeric matrix of predictors, or `mmap_matrix`.
 D Numeric matrix of dummies.
 y Numeric response vector.
 lambda2 Numeric L2 (ridge) penalty parameter (≥ 0).
 normalize Logical; column-normalize X and D (default TRUE).
 intercept Logical; fit an intercept (default TRUE).
 verbose Logical; print progress (default FALSE).

Method `get_num_removals()`: Get the number of coefficient removals.

Usage:

```
TENETSolver$get_num_removals()
```

Returns: Integer count of removals.

Method `get_cycling_ratio()`: Get the cycling ratio (removals / total steps).

Usage:

```
TENETSolver$get_cycling_ratio()
```

Returns: Scalar numeric cycling ratio.

Method `get_lambda()`: Get the regularization (lambda) path.

Usage:

```
TENETSolver$get_lambda()
```

Returns: Numeric vector of lambda values, one per step.

Method `clone()`: The objects of this class are cloneable with this method.

Usage:

```
TENETSolver$clone(deep = FALSE)
```

Arguments:

deep Whether to make a deep clone.

TGPSolver	<i>T-GP Solver</i>
-----------	--------------------

Description

Terminating Gradient Pursuit (T-GP) solver. Inherits all common methods from [TSolverBase](#).

Super class

[TRexSelector::TSolverBase](#) -> TGP Solver

Methods**Public methods:**

- [TGPSolver\\$new\(\)](#)
- [TGPSolver\\$clone\(\)](#)

Method [new\(\)](#): Create a new TGP Solver.

Usage:

```
TGPSolver$new(X, D, y, normalize = TRUE, intercept = TRUE, verbose = FALSE)
```

Arguments:

X Numeric matrix of predictors, or `mmap_matrix`.

D Numeric matrix of dummies.

y Numeric response vector.

normalize Logical; column-normalize X and D (default TRUE).

intercept Logical; fit an intercept (default TRUE).

verbose Logical; print progress (default FALSE).

Method [clone\(\)](#): The objects of this class are cloneable with this method.

Usage:

```
TGPSolver$clone(deep = FALSE)
```

Arguments:

deep Whether to make a deep clone.

TLARSSolver	<i>T-LARS Solver</i>
-------------	----------------------

Description

Terminating Least Angle Regression (T-LARS) solver. Inherits all common methods from [TSolverBase](#).

Super class

[TRexSelector::TSolverBase](#) -> TLARSSolver

Methods

Public methods:

- [TLARSSolver\\$new\(\)](#)
- [TLARSSolver\\$get_lambda\(\)](#)
- [TLARSSolver\\$clone\(\)](#)

Method `new()`: Create a new TLARSSolver.

Usage:

```
TLARSSolver$new(X, D, y, normalize = TRUE, intercept = TRUE, verbose = FALSE)
```

Arguments:

X Numeric matrix of predictors, or `mmap_matrix`.

D Numeric matrix of dummies.

y Numeric response vector.

normalize Logical; column-normalize X and D (default TRUE).

intercept Logical; fit an intercept (default TRUE).

verbose Logical; print progress (default FALSE).

Method `get_lambda()`: Get the regularization (lambda) path.

Usage:

```
TLARSSolver$get_lambda()
```

Returns: Numeric vector of lambda values, one per step.

Method `clone()`: The objects of this class are cloneable with this method.

Usage:

```
TLARSSolver$clone(deep = FALSE)
```

Arguments:

deep Whether to make a deep clone.

TLASSOSolver

T-LASSO Solver

Description

Terminating LASSO solver (T-LARS with variable removal). Inherits all common methods from [TSolverBase](#).

Super class

[TRexSelector::TSolverBase](#) -> TLASSOSolver

Methods

Public methods:

- [TLASSOSolver\\$new\(\)](#)
- [TLASSOSolver\\$get_num_removals\(\)](#)
- [TLASSOSolver\\$get_lambda\(\)](#)
- [TLASSOSolver\\$clone\(\)](#)

Method `new()`: Create a new TLASSOSolver.

Usage:

```
TLASSOSolver$new(X, D, y, normalize = TRUE, intercept = TRUE, verbose = FALSE)
```

Arguments:

X Numeric matrix of predictors, or `mmap_matrix`.

D Numeric matrix of dummies.

y Numeric response vector.

normalize Logical; column-normalize X and D (default TRUE).

intercept Logical; fit an intercept (default TRUE).

verbose Logical; print progress (default FALSE).

Method `get_num_removals()`: Get the number of coefficient removals.

Usage:

```
TLASSOSolver$get_num_removals()
```

Returns: Integer count of removals.

Method `get_lambda()`: Get the regularization (lambda) path.

Usage:

```
TLASSOSolver$get_lambda()
```

Returns: Numeric vector of lambda values, one per step.

Method `clone()`: The objects of this class are cloneable with this method.

Usage:

```
TLASSOSolver$clone(deep = FALSE)
```

Arguments:

deep Whether to make a deep clone.

TMPSolver

T-MP Solver

Description

Terminating Matching Pursuit (T-MP) solver. Inherits all common methods from [TSolverBase](#).

Super class

[TRexSelector::TSolverBase](#) -> TMPSolver

Methods

Public methods:

- [TMPSolver\\$new\(\)](#)
- [TMPSolver\\$clone\(\)](#)

Method `new()`: Create a new TMPSolver.

Usage:

```
TMPSolver$new(X, D, y, normalize = TRUE, intercept = TRUE, verbose = FALSE)
```

Arguments:

X Numeric matrix of predictors, or `mmap_matrix`.

D Numeric matrix of dummies.

y Numeric response vector.

normalize Logical; column-normalize X and D (default TRUE).

intercept Logical; fit an intercept (default TRUE).

verbose Logical; print progress (default FALSE).

Method `clone()`: The objects of this class are cloneable with this method.

Usage:

```
TMPSolver$clone(deep = FALSE)
```

Arguments:

deep Whether to make a deep clone.

TNCGMPSolver

T-NCGMP Solver

Description

Terminating Norm-Corrected Gradient Matching Pursuit (T-NCGMP) solver. Two variants are available: "line_search" (standard, 1-D projection) and "fully_corrective" (OMP-style least squares over the active set). Inherits all common methods from [TSolverBase](#).

Super class

[TRexSelector](#): [TSolverBase](#) -> TNCGMPSolver

Methods

Public methods:

- [TNCGMPSolver\\$new\(\)](#)
- [TNCGMPSolver\\$clone\(\)](#)

Method `new()`: Create a new TNCGMPSolver.

Usage:


```
TNCGMPSolver$new(
  X,
  D,
  y,
  variant = "line_search",
  normalize = TRUE,
  intercept = TRUE,
  verbose = FALSE
)
```

Arguments:

X Numeric matrix of predictors, or `mmap_matrix`.

D Numeric matrix of dummies.

y Numeric response vector.

variant Character; "line_search" (default) or "fully_corrective".

normalize Logical; column-normalize X and D (default TRUE).

intercept Logical; fit an intercept (default TRUE).

verbose Logical; print progress (default FALSE).

Method `clone()`: The objects of this class are cloneable with this method.

Usage:

```
TNCGMPSolver$clone(deep = FALSE)
```

Arguments:

deep Whether to make a deep clone.

TOMPSolver

T-OMP Solver

Description

Terminating Orthogonal Matching Pursuit (T-OMP) solver. Inherits all common methods from [TSolverBase](#).

Super class

[TRexSelector::TSolverBase](#) -> TOMPSolver

Methods**Public methods:**

- [TOMPSolver\\$new\(\)](#)
- [TOMPSolver\\$clone\(\)](#)

Method `new()`: Create a new TOMPSolver.

Usage:

```
TOMPSolver$new(X, D, y, normalize = TRUE, intercept = TRUE, verbose = FALSE)
```

Arguments:

X Numeric matrix of predictors, or `mmap_matrix`.

D Numeric matrix of dummies.
 y Numeric response vector.
 normalize Logical; column-normalize X and D (default TRUE).
 intercept Logical; fit an intercept (default TRUE).
 verbose Logical; print progress (default FALSE).

Method clone(): The objects of this class are cloneable with this method.

Usage:

TOOPLSSolver\$clone(deep = FALSE)

Arguments:

deep Whether to make a deep clone.

TOOLSSolver	<i>T-OLS Solver</i>
-------------	---------------------

Description

Terminating Orthogonal Least Squares (T-OLS) solver. Inherits all common methods from [TSolverBase](#).

Super class

[TRexSelector::TSolverBase](#) -> TOOLSSolver

Methods

Public methods:

- [TOOLSSolver\\$new\(\)](#)
- [TOOLSSolver\\$clone\(\)](#)

Method new(): Create a new TOOLSSolver.

Usage:

TOOLSSolver\$new(X, D, y, normalize = TRUE, intercept = TRUE, verbose = FALSE)

Arguments:

X Numeric matrix of predictors, or `mmap_matrix`.
 D Numeric matrix of dummies.
 y Numeric response vector.
 normalize Logical; column-normalize X and D (default TRUE).
 intercept Logical; fit an intercept (default TRUE).
 verbose Logical; print progress (default FALSE).

Method clone(): The objects of this class are cloneable with this method.

Usage:

TOOLSSolver\$clone(deep = FALSE)

Arguments:

deep Whether to make a deep clone.

TRexBiobankScreeningSelector

Biobank Screen T-Rex Selector

Description

Genomic biobank phenotype screening algorithm based on TRex architecture.

Methods

Public methods:

- `TRexBiobankScreeningSelector$new()`
- `TRexBiobankScreeningSelector$select()`
- `TRexBiobankScreeningSelector$clone()`

Method `new()`: Create a new TRexBiobankScreeningSelector object.

Usage:

```
TRexBiobankScreeningSelector$new(
  X,
  Y,
  target_FDR_trex = 0.1,
  lower_bound_FDR = 0.05,
  upper_bound_FDR = 0.15,
  use_bootstrap_CI = FALSE,
  R_boot = 1000,
  ci_grid_step = 0.001,
  trex_method = "TREX",
  method = "TLARS",
  K = 20,
  use_openmp = TRUE,
  use_memory_mapping = FALSE,
  max_outer_threads = 1,
  max_inner_threads = 1,
  seed = -1,
  verbose = TRUE
)
```

Arguments:

X Feature matrix.

Y Response vector (1D) or matrix (2D).

target_FDR_trex Target FDR for fallback T-Rex.

lower_bound_FDR Lower bound for acceptable estimated FDR.

upper_bound_FDR Upper bound for acceptable estimated FDR.

use_bootstrap_CI Whether to use bootstrap CI screening.

R_boot Number of bootstrap iterations.

ci_grid_step Grid step sizes for interval calculations.

trex_method Screening method: "TREX", "TREX_DA_AR1", "TREX_DA_EQUI", "TREX_DA_BLOCK_EQUI"

method The solver method, e.g., "TLARS", "TACGP", "TENET" (default: "TLARS").

K Number of random experiments (default: 20).

use_omp Whether to use OpenMP parallelization (default: TRUE).
 use_memory_mapping Use out-of-core memory mapping (default: FALSE).
 max_outer_threads Thread count for K repetitions (default: 1).
 max_inner_threads Thread count for inner solvers (default: 1).
 seed Random seed (default: -1).
 verbose Whether to print progress (default: TRUE).

Method `select()`: Run the Biobank Screening algorithm.

Usage:

```
TRexBiobankScreeningSelector$select()
```

Returns: A list with results. For vector 'Y', it gives a single phenotype result. For matrix 'Y', it gives a multi-phenotype result containing a \$statistics data.frame and \$selected_indices arrays.

Method `clone()`: The objects of this class are cloneable with this method.

Usage:

```
TRexBiobankScreeningSelector$clone(deep = FALSE)
```

Arguments:

deep Whether to make a deep clone.

TRexDASelector

Dependency-Aware T-Rex Selector (TRexDA)

Description

FDR-controlled variable selection algorithm for dependency scenarios.

Super class

```
TRexSelector : TRexSelector -> TRexDASelector
```

Active bindings

rho_thresh Optimal correlation threshold selected by the DA process.

rho_grid The grid of correlation thresholds evaluated.

da_method_used The integer code (or name mapped later) of the DA method used.

FDP_hat_array_BT A 3D array of FDP estimates for BT method: (Variables x Thresholds x K).

Phi_array_BT A 3D array of variables selected for BT method.

R_array_BT A 3D array mapping original to clustered features for BT method.

Methods

Public methods:

- [TRexDASelector\\$new\(\)](#)
- [TRexDASelector\\$select\(\)](#)
- [TRexDASelector\\$clone\(\)](#)

Method `new()`: Create a new TRexDASelector object.

Usage:

```
TRexDASelector$new(
  X,
  y,
  tFDR = 0.1,
  seed = -1,
  verbose = TRUE,
  method = "TLARS",
  da_method = "BT",
  cor_coef = -2,
  rho_thr_DA = 0.02,
  hc_linkage = "Single",
  hc_grid_length = 0,
  K = 20,
  max_dummy_multiplier = 10,
  use_max_T_stop = TRUE,
  opt_threshold = 0.75,
  lloop_strategy = "HCONCAT",
  tloop_stagnation_stop = TRUE,
  tloop_max_stagnant_steps = 5,
  use_openmp = TRUE,
  use_memory_mapping = FALSE,
  max_outer_threads = 1,
  max_inner_threads = 1,
  tol = 1e-06,
  lambda2 = 0.1,
  rho_afs = 0.3,
  ncgmp_variant = 0
)
```

Arguments:

`X` Feature matrix or MemoryMappedMatrix.

`y` Response vector.

`tFDR` Target FDR level (default: 0.1).

`seed` Random seed (default: -1).

`verbose` Whether to print progress (default: TRUE).

`method` The solver method, e.g., "TLARS", "TACGP", "TENET" (default: "TLARS").

`da_method` Dependency method: "BT", "AR1", "EQUI" (default: "BT").

`cor_coef` Correlation coefficient for AR1/EQUI.

`rho_thr_DA` Threshold below which equi-correlation is skipped (default: 0.02).

`hc_linkage` Hierarchical clustering linkage: "Single", "Complete", "Average", "WPGMA", "Ward" (default: "Single").

`hc_grid_length` Number of grid points for BT (default: 0).

K Number of random experiments (default: 20).
 max_dummy_multiplier Max dummy features multiplier (default: 10).
 use_max_T_stop Whether to bound max T_stop (default: TRUE).
 opt_threshold Optimization threshold (default: 0.75).
 lloop_strategy Strategy for L-loop (default: "HCONCAT").
 tloop_stagnation_stop Whether to stop early if stagnation (default: TRUE).
 tloop_max_stagnant_steps Steps before stagnation trigger (default: 5).
 use_openmp Whether to use OpenMP parallelization (default: TRUE).
 use_memory_mapping Use out-of-core memory mapping (default: FALSE).
 max_outer_threads Thread count for K repetitions (default: 1).
 max_inner_threads Thread count for inner solvers (default: 1).
 tol Tolerance for solver algorithms (default: 1e-6).
 lambda2 ENET hyperparameter (default: 0.1).
 rho_afs AFS hyperparameter (default: 0.3).
 ncgmp_variant NCGMP strategy (default: 0).

Method select(): Run the Dependency-Aware TRexSelector algorithm.

Usage:

TRexDASelector\$select()

Method clone(): The objects of this class are cloneable with this method.

Usage:

TRexDASelector\$clone(deep = FALSE)

Arguments:

deep Whether to make a deep clone.

TRexGVSSelector

T-Rex Selector with Group Variable Selection (TRex GVS)

Description

FDR-controlled variable selection algorithm utilizing group-based structural augmentation.

Super class

`TRexSelector::TRexSelector` -> TRexGVSSelector

Active bindings

lambda2_used Lambda2 regularization term selected/applied.
 gvs_type The augmentation policy name used (e.g., "EN", "IEN").
 max_clusters The number of unique clusters dynamically formulated.

Methods

Public methods:

- [TRexGVSSelector\\$new\(\)](#)
- [TRexGVSSelector\\$select\(\)](#)
- [TRexGVSSelector\\$clone\(\)](#)

Method `new()`: Create a new TRexGVSSelector object.

Usage:

```
TRexGVSSelector$new(
  X,
  y,
  tFDR = 0.1,
  seed = -1,
  verbose = TRUE,
  gvs_type = "EN",
  corr_max = 0.5,
  hc_linkage = "Single",
  lambda_2 = 0.1,
  groups = NULL,
  K = 20,
  max_dummy_multiplier = 10,
  use_max_T_stop = TRUE,
  opt_threshold = 0.75,
  lloop_strategy = "HCONCAT",
  tloop_stagnation_stop = TRUE,
  tloop_max_stagnant_steps = 5,
  use_openmp = TRUE,
  use_memory_mapping = FALSE,
  max_outer_threads = 1,
  max_inner_threads = 1
)
```

Arguments:

`X` Feature matrix or MemoryMappedMatrix.

`y` Response vector.

`tFDR` Target FDR level (default: 0.1).

`seed` Random seed (default: -1).

`verbose` Whether to print progress (default: TRUE).

`gvs_type` Augmentation policy: "EN", "IEN" (default: "EN").

`corr_max` Maximum pairwise correlation for clustering (default: 0.5).

`hc_linkage` Hierarchical clustering linkage method: "Single", "Complete", "Average", "WPGMA" (default: "Single").

`lambda_2` Lambda2 for grouping (default: 0.1).

`groups` User-specified groups (default: NULL).

`K` Number of random experiments (default: 20).

`max_dummy_multiplier` Max dummy features multiplier (default: 10).

`use_max_T_stop` Whether to bound max T_stop (default: TRUE).

`opt_threshold` Optimization threshold (default: 0.75).

`llloop_strategy` Strategy for L-loop (default: "HCONCAT").

tloop_stagnation_stop Whether to stop early if stagnation (default: TRUE).
 tloop_max_stagnant_steps Steps before stagnation trigger (default: 5).
 use_openmp Whether to use OpenMP parallelization (default: TRUE).
 use_memory_mapping Use out-of-core memory mapping (default: FALSE).
 max_outer_threads Thread count for K repetitions (default: 1).
 max_inner_threads Thread count for inner solvers (default: 1).

Method select(): Run the GVS algorithm.

Usage:

```
TRexGVSSelector$select()
```

Method clone(): The objects of this class are cloneable with this method.

Usage:

```
TRexGVSSelector$clone(deep = FALSE)
```

Arguments:

deep Whether to make a deep clone.

TRexScreeningSelector *Screen T-Rex Selector*

Description

Fast variable screening algorithm based on TRex architecture.

Super class

```
TRexSelector::TRexSelector -> TRexScreeningSelector
```

Active bindings

estimated_FDR Empirical FDR expectation bounds.
 used_bootstrap Employs bootstrapping.
 confidence_level Bounds the bootstrap estimates.
 estimated_correlation Covariance max value.
 beta_mat Complete variables parameter space.
 dummy_betas Dummy variables parameter space.

Methods

Public methods:

- `TRexScreeningSelector$new()`
- `TRexScreeningSelector$select()`
- `TRexScreeningSelector$clone()`

Method new(): Create a new TRexScreeningSelector object.

Usage:


```

TRexScreeningSelector$new(
  X,
  y,
  seed = -1,
  verbose = TRUE,
  use_bootstrap_CI = FALSE,
  R_boot = 1000,
  ci_grid_step = 0.001,
  trex_method = "TREX",
  method = "TLARS",
  K = 20,
  use_openmp = TRUE,
  use_memory_mapping = FALSE,
  max_outer_threads = 1,
  max_inner_threads = 1
)

```

Arguments:

X Feature matrix or MemoryMappedMatrix.

y Response vector.

seed Random seed (default: -1).

verbose Whether to print progress (default: TRUE).

use_bootstrap_CI Whether to use bootstrap CI screening.

R_boot Number of bootstrap iterations.

ci_grid_step the grid step sizes for interval calculations

trex_method Screening method: "TREX", "TREX_DA_AR1", "TREX_DA_EQUI", "TREX_DA_BLOCK_EQUI"

method The solver method, e.g., "TLARS", "TACGP", "TENET" (default: "TLARS").

K Number of random experiments (default: 20).

use_openmp Whether to use OpenMP parallelization (default: TRUE).

use_memory_mapping Use out-of-core memory mapping (default: FALSE).

max_outer_threads Thread count for K repetitions (default: 1).

max_inner_threads Thread count for inner solvers (default: 1).

Method select(): Run the Screening algorithm.

Usage:

```
TRexScreeningSelector$select()
```

Method clone(): The objects of this class are cloneable with this method.

Usage:

```
TRexScreeningSelector$clone(deep = FALSE)
```

Arguments:

deep Whether to make a deep clone.

TRexSelector

T-Rex Selector

Description

FDR-controlled variable selection algorithm.

Active bindings

.ptr Get internal pointer.
 .ptr_X internal proxy for data size
 selected_var Logical vector indicating which variables were selected.
 selected_indices Integer vector of selected 1-based indices.
 v_thresh The chosen voting threshold.
 T_stop The length of the path / maximum number of included variables.
 L The number of dummy replications used.
 voting_grid The grid of voters over trials and variables.
 fdp_hat_mat estimated false discovery proportions
 phi_mat probability mass function evaluations
 r_mat empirical CDF computations
 phi_prime smoothed probability masses
 x_means computed means of the predictor variables
 x_l2_norms computed L2 norms of the scaled predictors
 y_mean computed mean of the response sequence

Methods

Public methods:

- [TRexSelector\\$new\(\)](#)
- [TRexSelector\\$select\(\)](#)
- [TRexSelector\\$clone\(\)](#)

Method `new()`: Create a new TRexSelector object.

Usage:

```
TRexSelector$new(
  X,
  y,
  tFDR = 0.1,
  seed = -1,
  verbose = TRUE,
  method = "TLARS",
  K = 20,
  max_dummy_multiplier = 10,
  use_max_T_stop = TRUE,
  opt_threshold = 0.75,
  lloop_strategy = "HCONCAT",
```

```

        tloop_stagnation_stop = FALSE,
        tloop_max_stagnant_steps = 5,
        use_openmp = FALSE,
        use_memory_mapping = FALSE,
        max_outer_threads = 1,
        max_inner_threads = 1,
        tol = 1e-06,
        lambda2 = 0.1,
        rho_afs = 0.3,
        ncgmp_variant = 0
    )

```

Arguments:

X Feature matrix or MemoryMappedMatrix.

y Response vector.

tFDR Target FDR level (default: 0.1).

seed Random seed (default: -1).

verbose Whether to print progress (default: TRUE).

method The solver method, e.g., "TLARS", "TACGP", "TENET" (default: "TLARS").

K Number of random experiments (default: 20).

max_dummy_multiplier Max dummy features multiplier (default: 10).

use_max_T_stop Whether to bound max T_stop (default: TRUE).

opt_threshold Optimization threshold (default: 0.75).

lloop_strategy Strategy for L-loop (default: "HCONCAT").

tloop_stagnation_stop Whether to stop early if stagnation (default: FALSE).

tloop_max_stagnant_steps Steps before stagnation trigger (default: 5).

use_openmp Whether to use OpenMP parallelization (default: FALSE).

use_memory_mapping Use out-of-core memory mapping (default: FALSE).

max_outer_threads Thread count for K repetitions (default: 1).

max_inner_threads Thread count for inner solvers (default: 1).

tol Tolerance for solver algorithms (default: 1e-6).

lambda2 ENET hyperparameter (default: 0.1).

rho_afs AFS hyperparameter (default: 0.3).

ncgmp_variant NCGMP strategy (default: 0).

Method `select()`: Run the TRexSelector algorithm.

Usage:

```
TRexSelector$select()
```

Returns: invisible(self)

Method `clone()`: The objects of this class are cloneable with this method.

Usage:

```
TRexSelector$clone(deep = FALSE)
```

Arguments:

deep Whether to make a deep clone.

TSolverBase

*Base T-Solver Class***Description**

Abstract base class for all T-Solver R6 wrappers. Provides the common solver API backed by a C++ TSolver_Base instance. Users should instantiate one of the concrete subclasses (TLARSSolver, TLASSOSolver, etc.) rather than this class directly.

Public fields

X Numeric matrix of predictors, or an `mmap_matrix`.
 D Numeric matrix of dummies.
 y Numeric response vector.
 ptr External pointer to the underlying C++ TSolver_Base object.

Methods**Public methods:**

- `TSolverBase$new()`
- `TSolverBase$execute_step()`
- `TSolverBase$get_num_steps()`
- `TSolverBase$get_actions()`
- `TSolverBase$get_beta()`
- `TSolverBase$get_cp()`
- `TSolverBase$get_rss()`
- `TSolverBase$get_r2()`
- `TSolverBase$get_dof()`
- `TSolverBase$get_actives()`
- `TSolverBase$get_inactives()`
- `TSolverBase$get_intercept()`
- `TSolverBase$save()`
- `TSolverBase$load()`
- `TSolverBase$clone()`

Method `new()`: Validate and store data fields. Called by every subclass initialize. Not intended for direct instantiation.

Usage:

`TSolverBase$new(X, D, y)`

Arguments:

X Numeric matrix of predictors, or `mmap_matrix`.
 D Numeric matrix of dummies.
 y Numeric response vector.

Method `execute_step()`: Execute solver step.

Usage:

```
TSolverBase$execute_step(T_stop = -1, early_stop = TRUE)
```

Arguments:

`T_stop` Integer stopping criterion; -1 runs until the early-stop condition is met.

`early_stop` Logical; stop early when enough dummies are active.

Returns: Invisibly returns self for method chaining.

Method `get_num_steps()`: Get the number of steps executed so far.

Usage:

```
TSolverBase$get_num_steps()
```

Returns: Integer number of completed steps.

Method `get_actions()`: Get the action path (variable additions and removals per step).

Usage:

```
TSolverBase$get_actions(r_index = TRUE)
```

Arguments:

`r_index` Logical; if TRUE (default) indices inside each action vector are 1-based (R convention); if FALSE they are 0-based (C++ convention).

Returns: A list of integer vectors, one per step. Positive indices denote variable additions; negative indices denote removals.

Method `get_beta()`: Get coefficient vector at a given step.

Usage:

```
TSolverBase$get_beta(step = -1)
```

Arguments:

`step` Integer step index; -1 returns the current/final step.

Returns: Numeric vector of coefficients.

Method `get_cp()`: Get Mallows Cp scores along the solution path.

Usage:

```
TSolverBase$get_cp()
```

Returns: Numeric vector of Cp scores, one per step.

Method `get_rss()`: Get residual sum of squares along the solution path.

Usage:

```
TSolverBase$get_rss()
```

Returns: Numeric vector of RSS values, one per step.

Method `get_r2()`: Get R-squared values along the solution path.

Usage:

```
TSolverBase$get_r2()
```

Returns: Numeric vector of R-squared values, one per step.

Method `get_dof()`: Get degrees of freedom along the solution path.

Usage:

```
TSolverBase$get_dof()
```

Returns: Numeric vector of degrees of freedom, one per step.

Method `get_actives()`: Get currently active variable indices.

Usage:

```
TSolverBase$get_actives(r_index = TRUE)
```

Arguments:

`r_index` Logical; if TRUE (default) indices are 1-based (R convention); if FALSE they are 0-based (C++ convention).

Returns: Integer vector of active variable indices.

Method `get_inactives()`: Get currently inactive variable indices.

Usage:

```
TSolverBase$get_inactives(r_index = TRUE)
```

Arguments:

`r_index` Logical; if TRUE (default) indices are 1-based (R convention); if FALSE they are 0-based (C++ convention).

Returns: Integer vector of inactive variable indices.

Method `get_intercept()`: Get the fitted intercept at a given step.

Usage:

```
TSolverBase$get_intercept(step = -1)
```

Arguments:

`step` Integer step index; -1 returns the current/final step.

Returns: Scalar numeric intercept.

Method `save()`: Serialize the solver state to a binary file.

Usage:

```
TSolverBase$save(filename)
```

Arguments:

`filename` Character; path to the output file.

Returns: Invisibly returns self.

Method `load()`: Restore the solver state from a binary file created by `$save()`.

Usage:

```
TSolverBase$load(filename)
```

Arguments:

`filename` Character; path to the input file.

Returns: Invisibly returns self.

Method `clone()`: The objects of this class are cloneable with this method.

Usage:

```
TSolverBase$clone(deep = FALSE)
```

Arguments:

`deep` Whether to make a deep clone.

TStagewiseSolver	<i>T-Stagewise Solver</i>
------------------	---------------------------

Description

Terminating Stagewise Regression solver. Inherits all common methods from [TSolverBase](#).

Super class

[TRexSelector](#): [TSolverBase](#) -> TStagewiseSolver

Methods

Public methods:

- [TStagewiseSolver\\$new\(\)](#)
- [TStagewiseSolver\\$get_num_removals\(\)](#)
- [TStagewiseSolver\\$get_cycling_ratio\(\)](#)
- [TStagewiseSolver\\$get_lambda\(\)](#)
- [TStagewiseSolver\\$clone\(\)](#)

Method [new\(\)](#): Create a new TStagewiseSolver.

Usage:

```
TStagewiseSolver$new(
  X,
  D,
  y,
  normalize = TRUE,
  intercept = TRUE,
  verbose = FALSE
)
```

Arguments:

X Numeric matrix of predictors, or `mmap_matrix`.
D Numeric matrix of dummies.
y Numeric response vector.
normalize Logical; column-normalize X and D (default TRUE).
intercept Logical; fit an intercept (default TRUE).
verbose Logical; print progress (default FALSE).

Method [get_num_removals\(\)](#): Get the number of coefficient removals.

Usage:

```
TStagewiseSolver$get_num_removals()
```

Returns: Integer count of removals.

Method [get_cycling_ratio\(\)](#): Get the cycling ratio (removals / total steps).

Usage:

```
TStagewiseSolver$get_cycling_ratio()
```

Returns: Scalar numeric cycling ratio.

Method `get_lambda()`: Get the regularization (lambda) path.

Usage:

```
TStagewiseSolver$get_lambda()
```

Returns: Numeric vector of lambda values, one per step.

Method `clone()`: The objects of this class are cloneable with this method.

Usage:

```
TStagewiseSolver$clone(deep = FALSE)
```

Arguments:

`deep` Whether to make a deep clone.

TSTEPWISESolver

T-Stepwise Solver

Description

Terminating Forward Stepwise Regression solver. Inherits all common methods from [TSolverBase](#).

Super class

[TReXSelector::TSolverBase](#) -> TSTEPWISESolver

Methods

Public methods:

- [TSTEPWISESolver\\$new\(\)](#)
- [TSTEPWISESolver\\$get_lambda\(\)](#)
- [TSTEPWISESolver\\$clone\(\)](#)

Method `new()`: Create a new TSTEPWISESolver.

Usage:

```
TSTEPWISESolver$new(
  X,
  D,
  y,
  normalize = TRUE,
  intercept = TRUE,
  verbose = FALSE
)
```

Arguments:

`X` Numeric matrix of predictors, or `mmap_matrix`.

`D` Numeric matrix of dummies.

`y` Numeric response vector.

`normalize` Logical; column-normalize X and D (default TRUE).

`intercept` Logical; fit an intercept (default TRUE).

`verbose` Logical; print progress (default FALSE).

Method `get_lambda()`: Get the regularization (lambda) path.

Usage:

`TSTEPWISESolver$get_lambda()`

Returns: Numeric vector of lambda values, one per step.

Method `clone()`: The objects of this class are cloneable with this method.

Usage:

`TSTEPWISESolver$clone(deep = FALSE)`

Arguments:

`deep` Whether to make a deep clone.

ZScoreScaler

Z-Score Scaler

Description

Standardizes features by removing the mean and scaling to unit variance.

Methods

Public methods:

- `ZScoreScaler$new()`
- `ZScoreScaler$fit()`
- `ZScoreScaler$transform_inplace()`
- `ZScoreScaler$inverse_transform_inplace()`
- `ZScoreScaler$is_fitted()`
- `ZScoreScaler$get_dropped_indices()`
- `ZScoreScaler$get_with_mean()`
- `ZScoreScaler$get_with_std()`
- `ZScoreScaler$get_means()`
- `ZScoreScaler$get_scales()`
- `ZScoreScaler$save()`
- `ZScoreScaler$load()`
- `ZScoreScaler$clone()`

Method `new()`: Initialize the ZScoreScaler

Usage:

`ZScoreScaler$new(with_mean = TRUE, with_std = TRUE)`

Arguments:

`with_mean` Logical, whether to center the data before scaling (default: TRUE)

`with_std` Logical, whether to scale the data to unit variance (default: TRUE)

Method `fit()`: Compute the mean and std to be used for later scaling.

Usage:

`ZScoreScaler$fit(X, threshold = 1e-12)`

Arguments:

X A numeric matrix.

threshold Double, lower bound for standard deviation (default: 1e-12).

Returns: Value.

Method `transform_inplace()`: Perform standardization by centering and scaling in-place.

Usage:

```
ZScoreScaler$transform_inplace(X)
```

Arguments:

X A numeric matrix.

Returns: The modified matrix *X*.

Method `inverse_transform_inplace()`: Inverse transform to original scale.

Usage:

```
ZScoreScaler$inverse_transform_inplace(X)
```

Arguments:

X A numeric matrix.

Returns: The modified matrix *X*.

Method `is_fitted()`: Check if scaler is fitted.

Usage:

```
ZScoreScaler$is_fitted()
```

Returns: Value.

Method `get_dropped_indices()`: Get dropped indices.

Usage:

```
ZScoreScaler$get_dropped_indices()
```

Returns: Value.

Method `get_with_mean()`: Check if scaler centers data.

Usage:

```
ZScoreScaler$get_with_mean()
```

Returns: Value.

Method `get_with_std()`: Check if scaler scales data to unit variance.

Usage:

```
ZScoreScaler$get_with_std()
```

Returns: Value.

Method `get_means()`: Get the learned means.

Usage:

```
ZScoreScaler$get_means()
```

Returns: Value.

Method `get_scales()`: Get the learned scales (standard deviations).

Usage:

```
ZScoreScaler$get_scales()
```

Returns: Value.

Method `save()`: Save model state to file.

Usage:

```
ZScoreScaler$save(filename)
```

Arguments:

`filename` String path to save output.

Returns: Value.

Method `load()`: Load model state from file.

Usage:

```
ZScoreScaler$load(filename)
```

Arguments:

`filename` String path to load input from.

Returns: Value.

Method `clone()`: The objects of this class are cloneable with this method.

Usage:

```
ZScoreScaler$clone(deep = FALSE)
```

Arguments:

`deep` Whether to make a deep clone.

[.mmap_matrix

Extract block from mmap_matrix

Description

Extract block from mmap_matrix

Usage

```
## S3 method for class 'mmap_matrix'
x[i, j, ..., drop = FALSE]
```

Arguments

<code>x</code>	mmap_matrix object
<code>i</code>	row indices
<code>j</code>	column indices
<code>...</code>	unused arguments
<code>drop</code>	logical. If TRUE the result is coerced to the lowest possible dimension (not supported).

Value

a matrix block

Index

* datasets

DistanceMetric, 8

Gauss_data, 8

LinkageMethod, 9

[.mmap_matrix, 43

agglomerative_cluster, 2

as.matrix.mmap_matrix, 3

compute_fdp, 3

compute_fdp_dense, 4

compute_precision, 4

compute_recall, 5

compute_tpp, 5

compute_tpp_dense, 6

convert_to_memory_mapped, 6

cut_tree, 7

dim.mmap_matrix, 7

DistanceMetric, 8

Gauss_data, 8

get_ptr, 9

LinkageMethod, 9

LpNormScaler, 10

mmap_matrix, 12

print.mmap_matrix, 13

RidgeCV, 13

RidgeGCV, 15

TACGPSolver, 17

TAFSSolver, 18

TENETSolver, 19

TGPSolver, 21

TLARSSolver, 21

TLASSOSolver, 22

TMPSolver, 23

TNCGMPSolver, 24

TOMPSolver, 25

TOOLSSolver, 26

TRexBiobankScreeningSelector, 27

TRexDASelector, 28

TRexGVSSelector, 30

TRexScreeningSelector, 32

TRexSelector, 34

TRexSelector::TRexSelector, 28, 30, 32

TRexSelector::TSolverBase, 17–19, 21–26, 39, 40

TSolverBase, 17–19, 21–26, 36, 39, 40

TStagewiseSolver, 39

TSTEPWISESolver, 40

ZScoreScaler, 41